# lecture_32

October 25, 2026

## 1 Lecture 32: Ant Colony Optimization - Benchmarking

---

`{note} Lecture 31 ran ACOR - the continuous-domain bridge - on the Ackley function. This lecture calibrates ACO in its **native** graph-based form directly on a TSPLIB instance, the same pattern Lectures 26 and 29 used for Simulated Annealing and the Genetic Algorithm. Because calibration now happens on a genuinely combinatorial benchmark, there is no encoding bridge needed at all here - unlike ACOR's archive, native ACO's pheromone parameters are exactly what Lecture 33's Chennai Metro Feeder Route capstone will use.`

**Learning Objectives**

By the end of this notebook, you will be able to: 1. Implement native graph-based Ant Colony Optimization and apply it to a TSPLIB instance. 2. Calibrate the evaporation rate on a small instance under a limited budget. 3. Compare how ACO's calibration scales to a larger instance against Simulated Annealing's and the Genetic Algorithm's scaling behaviour (Lectures 26, 29).

**Prerequisites**: Ant Colony Optimization - Algorithm (Lecture 31); Genetic Algorithm - Benchmarking (Lecture 29).

**Estimated time**: 50 minutes

---

### 1.1 Native ACO on a TSPLIB Instance

Lecture 30 developed ACO's construction rule and pheromone update for a graph; Lecture 31 needed ACOR only because Ackley has no graph to construct on. `eil51` and `eil101` **are** graphs — every stop is a node, every pair of stops an edge — so native ACO, exactly as Lecture 30's pseudocode describes it, applies directly. No archive, no Gaussian sampling, no encoding bridge.

[1]:
```
import random
import numpy as np
import matplotlib.pyplot as plt

def aco(N, f, D, alpha, beta, rho, Q, m, n=200, t=1e-8, tau_o=1.0):
    """Ant Colony Optimization.

    Parameters
```

```
    ----------
    N     : customer nodes to visit (tours start and end at depot node 0)
    f     : objective function, f(s) -> float (minimized)
    D     : distance matrix, D[i, j] -> distance from node i to node j
    alpha : pheromone weight in the construction rule
    beta  : heuristic (1 / distance) weight in the construction rule
    rho   : pheromone evaporation rate
    Q     : pheromone deposit constant
    m     : number of ants per iteration
    n     : maximum number of iterations
    t     : convergence threshold on improvement in the best solution
    tau_o : initial pheromone level on every edge
    """
    V = [0] + list(N)
    tau = {(i, j): tau_o for i in V for j in V if i != j}             #␣
↪pheromone on each edge
    eta = {(i, j): 1.0 / D[i, j] for i in V for j in V if i != j}     #␣
↪heuristic desirability of each edge

    def construct():
        U, i, s = set(N), 0, []                  # unvisited nodes, current␣
↪node, partial tour
        while U:
            w = np.array([tau[(i, j)] ** alpha * eta[(i, j)] ** beta for j in␣
↪U])
            P = w / w.sum()                      # transition probabilities
            j = random.choices(list(U), weights=P, k=1)[0]
            s.append(j)
            U.discard(j)
            i = j
        return tuple(s)

    s_b = construct()                            # best solution found so far
    F_b = [f(s_b)]
    k, r, converged = 0, float("inf"), False
    while not converged:
        S = [construct() for _ in range(m)]      # each ant constructs a tour
        for key in tau:                          # evaporate
            tau[key] *= (1 - rho)
        for s in S:                              # deposit: Q / f(s) on every␣
↪edge of each tour
            T = [0] + list(s) + [0]
            for a, b in zip(T[:-1], T[1:]):
                tau[(a, b)] += Q / f(s)
                tau[(b, a)] += Q / f(s)
        s_c = min(S, key=f)                      # best solution in the current␣
↪iteration
```

```
        if f(s_c) < f(s_b):
            r = f(s_b) - f(s_c)
            s_b = s_c
        F_b.append(f(s_b))
        k += 1
        if k >= n or r <= t:
            converged = True
    return s_b, F_b
```

[2]:
```
def f(s):
    """Return the tour length for a permutation `s` of customer indices.

    This function assumes a global distance matrix `D` is defined, with
    `D[i, j]` giving the distance between city `i` and city `j`. The tour is
    interpreted as: depot -> s[0] -> ... -> s[-1] -> depot.

    Parameters
    ----------
    s : sequence
        A tour represented as a permutation of customer indices.

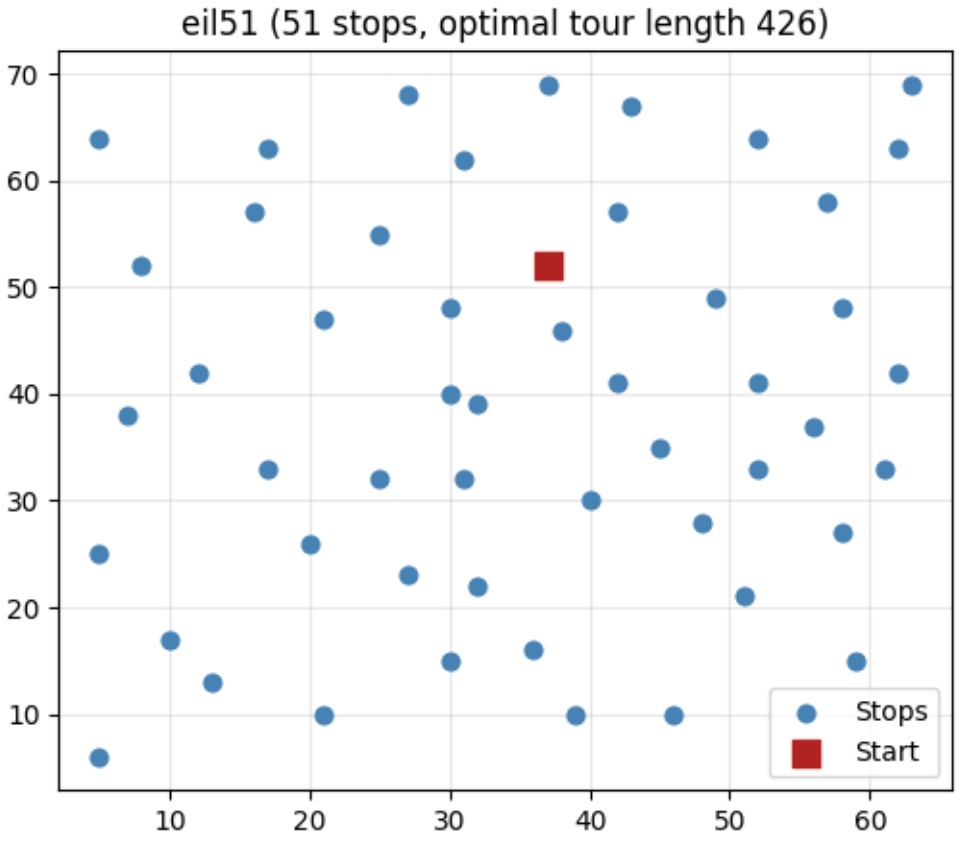
    Returns
    -------
    float
        Total travel cost of the tour.
    """
    c = D[0, s[0]]
    for i in range(len(s) - 1):
        c += D[s[i], s[i + 1]]
    c += D[s[-1], 0]
    return c
```

[3]:
```
# eil51: 51-city problem
coords = [
    (37, 52), (49, 49), (52, 64), (20, 26), (40, 30), (21, 47), (17, 63), (31,␣
 ↪62), (52, 33), (51, 21),
    (42, 41), (31, 32), (5, 25), (12, 42), (36, 16), (52, 41), (27, 23), (17,␣
 ↪33), (13, 13), (57, 58),
    (62, 42), (42, 57), (16, 57), (8, 52), (7, 38), (27, 68), (30, 48), (43,␣
 ↪67), (58, 48), (58, 27),
    (37, 69), (38, 46), (46, 10), (61, 33), (62, 63), (63, 69), (32, 22), (45,␣
 ↪35), (59, 15), (5, 6),
    (10, 17), (21, 10), (5, 64), (30, 15), (39, 10), (32, 39), (25, 32), (25,␣
 ↪55), (48, 28), (56, 37),
    (30, 40),
]
depot = 0
```

```
customers = list(range(1, len(coords)))
optimal = 426

n = len(coords)
X = np.array([c[0] for c in coords], dtype=float)
Y = np.array([c[1] for c in coords], dtype=float)
D = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        D[i, j] = np.hypot(X[i] - X[j], Y[i] - Y[j])

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(X[1:], Y[1:], c="steelblue", s=40, zorder=2, label="Stops")
ax.scatter(X[0], Y[0], c="firebrick", marker="s", s=100, zorder=3,␣
 ↪label="Start")
ax.set_title("eil51 (51 stops, optimal tour length 426)")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



### 1.2 Calibrating the Evaporation Rate

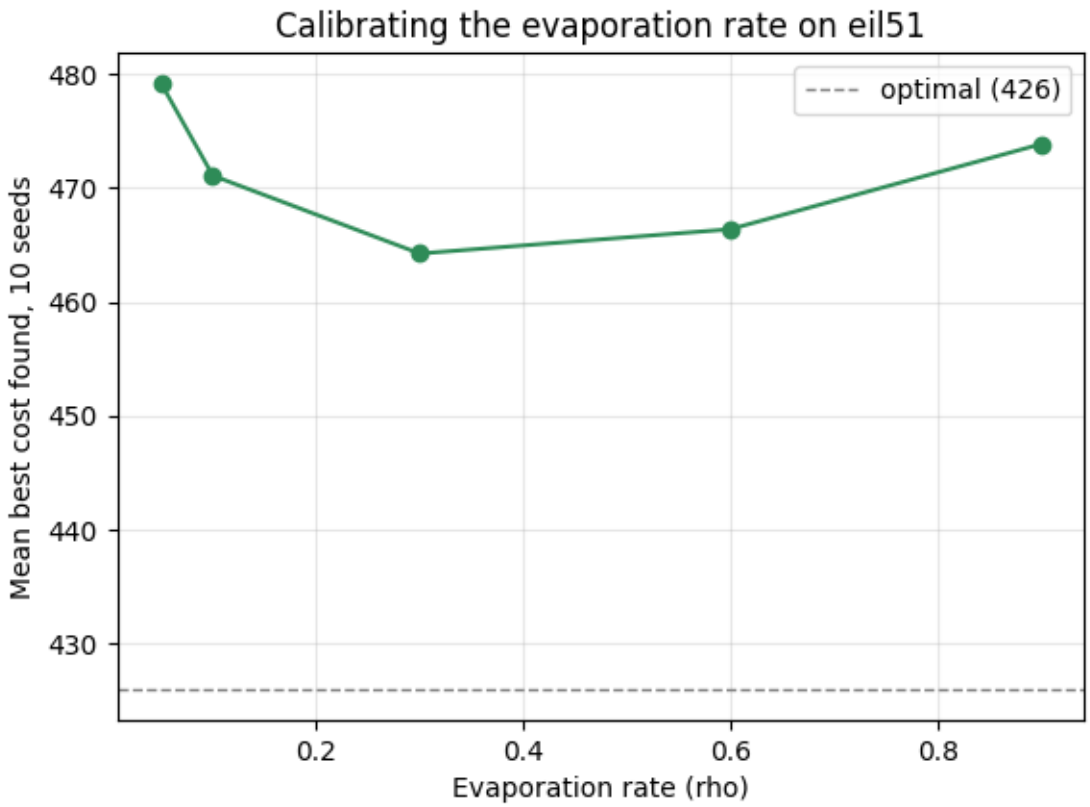
Fixing $\alpha = 1$, $\beta = 3$, $Q = 10$, $m = 10$ ants, a 100-iteration budget, averaged over 10 seeds — only $\rho$ changes.

[4]:
```
def run_aco(rho, alpha=1, beta=3, Q=10, m=10, n_iters=100, n_seeds=10, D=D,␣
 ↪customers=customers):
    results = []
    for seed in range(1, n_seeds + 1):
        random.seed(seed)
        np.random.seed(seed)
        s_b, F_b = aco(customers, f, D, alpha, beta, rho, Q, m, n=n_iters,␣
 ↪t=1e-9)
        results.append(f(s_b))
    return np.array(results)

rho_results = {}
for rho in [0.05, 0.1, 0.3, 0.6, 0.9]:
    vals = run_aco(rho)
    rho_results[rho] = vals
    gap = (vals.mean() - optimal) / optimal * 100
    print(f"rho={rho:<5} -> mean cost = {vals.mean():.2f}   gap = {gap:.1f}%")
```

```
rho=0.05  -> mean cost = 479.18   gap = 12.5%
rho=0.1   -> mean cost = 471.07   gap = 10.6%
rho=0.3   -> mean cost = 464.23   gap = 9.0%
rho=0.6   -> mean cost = 466.37   gap = 9.5%
rho=0.9   -> mean cost = 473.83   gap = 11.2%
```

[5]:
```
fig, ax = plt.subplots(figsize=(6.5, 4.5))
ax.plot(list(rho_results.keys()), [v.mean() for v in rho_results.values()],␣
 ↪"o-", color="seagreen")
ax.axhline(optimal, color="gray", linestyle="--", linewidth=1, label=f"optimal␣
 ↪({optimal})")
ax.set_xlabel("Evaporation rate (rho)")
ax.set_ylabel("Mean best cost found, 10 seeds")
ax.set_title("Calibrating the evaporation rate on eil51")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```




{note} $\rho=0.3$ is the clear best, with both lower and higher evaporation performing worse - the same interior sweet-spot shape Lecture 26 found for Simulated Annealing's cooling rate: too little evaporation lets early, possibly-lucky pheromone dominate construction indefinitely; too much forgets useful trails before they can accumulate.

---

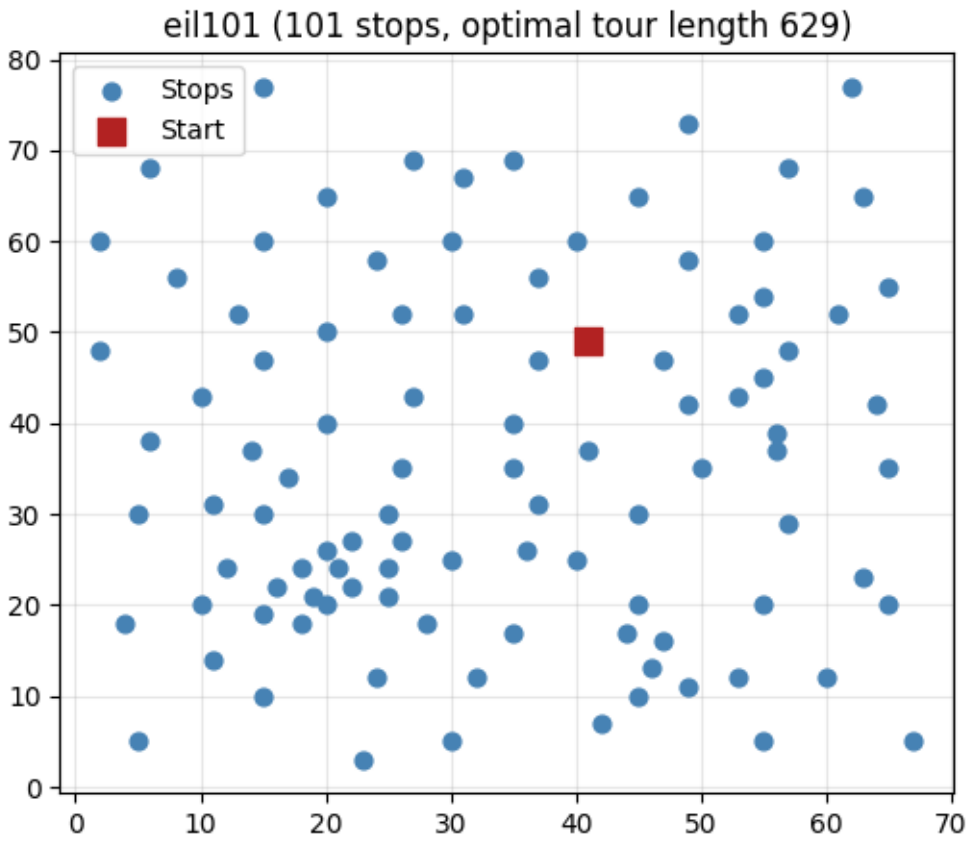
### 1.3 Benchmarking at Scale

[6]:
```
# eil101: 101-city problem
coords = [
    (41, 49), (35, 17), (55, 45), (55, 20), (15, 30), (25, 30), (20, 50), (10,␣
 ↪43), (55, 60), (30, 60),
    (20, 65), (50, 35), (30, 25), (15, 10), (30, 5), (10, 20), (5, 30), (20,␣
 ↪40), (15, 60), (45, 65),
    (45, 20), (45, 10), (55, 5), (65, 35), (65, 20), (45, 30), (35, 40), (41,␣
 ↪37), (64, 42), (40, 60),
    (31, 52), (35, 69), (53, 52), (65, 55), (63, 65), (2, 60), (20, 20), (5,␣
 ↪5), (60, 12), (40, 25),
```

```
    (42, 7), (24, 12), (23, 3), (11, 14), (6, 38), (2, 48), (8, 56), (13, 52),␣
 ↪(6, 68), (47, 47),
    (49, 58), (27, 43), (37, 31), (57, 29), (63, 23), (53, 12), (32, 12), (36,␣
 ↪26), (21, 24), (17, 34),
    (12, 24), (24, 58), (27, 69), (15, 77), (62, 77), (49, 73), (67, 5), (56,␣
 ↪39), (37, 47), (37, 56),
    (57, 68), (47, 16), (44, 17), (46, 13), (49, 11), (49, 42), (53, 43), (61,␣
 ↪52), (57, 48), (56, 37),
    (55, 54), (15, 47), (14, 37), (11, 31), (16, 22), (4, 18), (28, 18), (26,␣
 ↪52), (26, 35), (31, 67),
    (15, 19), (22, 22), (18, 24), (26, 27), (25, 24), (22, 27), (25, 21), (19,␣
 ↪21), (20, 26), (18, 18),
    (35, 35),
]
depot = 0
customers = list(range(1, len(coords)))
optimal = 629

n = len(coords)
X = np.array([c[0] for c in coords], dtype=float)
Y = np.array([c[1] for c in coords], dtype=float)
D = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        D[i, j] = np.hypot(X[i] - X[j], Y[i] - Y[j])

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(X[1:], Y[1:], c="steelblue", s=40, zorder=2, label="Stops")
ax.scatter(X[0], Y[0], c="firebrick", marker="s", s=100, zorder=3,␣
 ↪label="Start")
ax.set_title("eil101 (101 stops, optimal tour length 629)")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```




[7]:
```
random.seed(42)
np.random.seed(42)
s_b, F_b = aco(customers, f, D, 1, 3, 0.3, 10, 10, n=100, t=1e-9)
cost = f(s_b)
print(f"Calibrated (rho=0.3), same budget m=10, n=100: cost = {cost:.2f}   "
      f"gap = {(cost - optimal) / optimal * 100:.1f}%")
```

```
Calibrated (rho=0.3), same budget m=10, n=100: cost = 739.15   gap = 17.5%
```

[8]:
```
for m, iters in [(10, 300), (20, 300)]:
    random.seed(42)
    np.random.seed(42)
    s_b, F_b = aco(customers, f, D, 1, 3, 0.3, 10, m, n=iters, t=1e-9)
    cost = f(s_b)
    print(f"m={m:<4} iters={iters:<5} -> cost = {cost:.2f}   gap = {(cost -␣
 ↪optimal) / optimal * 100:.1f}%")
```

```
m=10   iters=300   -> cost = 722.17   gap = 14.8%
m=20   iters=300   -> cost = 721.09   gap = 14.6%
```

**Interpretation.** ACO's calibrated evaporation rate carries over cleanly — even at the *same* budget

it was calibrated at, the gap on `eil101` is already the smallest of any algorithm's naive transfer in this module, and it improves further with modestly more ants or iterations, without needing $\rho$ itself to change character. Across all three Benchmarking lectures, a consistent ranking emerges: **ACO's calibration transferred best, the Genetic Algorithm's transferred moderately (needing more budget but not a different setting), and Simulated Annealing's required an explicit change to the parameter itself.**

> **Managerial insight**: this ranking is not an accident of tuning — it tracks each algorithm's exploration mechanism. SA's single search thread depends entirely on its temperature schedule to avoid getting stuck, so a schedule miscalibrated for the instance size fails hard. GA and ACO both maintain many candidates at once (a population, a colony's parallel constructions), which cushions a suboptimal parameter choice by simply exploring more broadly regardless. **Algorithms that hedge across many simultaneous candidates degrade more gracefully under scale than algorithms that commit to one search thread at a time** — a concrete, evidence-based version of the tradeoff Lecture 23's taxonomy first raised in the abstract.

---

## 1.4 Take-Away Exercises

### 1.4.1 Exercise 1 — The Heuristic Weight at Scale

Set $\beta = 0$ (pheromone only, no distance heuristic) and repeat this lecture's calibration and benchmarking-at-scale runs on `eil51` and `eil101`. Does removing the heuristic term hurt more, or less, at the larger scale than at the smaller one?

### 1.4.2 Exercise 2 — Colony Size vs. Iterations

Matching total ant-constructions ($m\times$ iterations) roughly constant, compare $m = 10/300$ iterations against $m = 30/100$ iterations on `eil101`. Does it matter which one you scale up, the way it didn't much matter for the Genetic Algorithm (Lecture 29, Take-Away Exercise 1)?

---

## 1.5 Circling Back

- **Lecture 31 (Ant Colony Optimization: Algorithm)**: applied ACOR's archive-based bridge to Ackley; `aco()` above is the *native* form Lecture 30's pseudocode always described, applied here for the first time in this arc, now that the module works with genuine routing graphs.
- **Lecture 26 (Simulated Annealing: Benchmarking)** and **Lecture 29 (Genetic Algorithm: Benchmarking)**: all three Benchmarking lectures ran the same calibrate-on-`eil51`, test-on-`eil101` experiment; the differences in how well each transferred are this module's clearest evidence for Lecture 23's taxonomy.

## 1.6 Moving Forward

- **Lecture 33 (Transportation Engineering Problems - V)**: the first time any of these three algorithms is applied to the Chennai Metro Feeder Route — directly at the capacitated level, bringing `sa()`, `ga()`, and `aco()` together for a head-to-head comparison.

## 1.7 Further Reading


- Reinelt, G. (1991). "TSPLIB — A Traveling Salesman Problem Library." *ORSA Journal on Computing*, 3(4), 376-384.
- Dorigo, M. and Stützle, T. (2004). *Ant Colony Optimization*. MIT Press.
- Stützle, T. and Hoos, H.H. (2000). "MAX-MIN Ant System." *Future Generation Computer Systems*, 16(8), 889-914.